Content-based Filtering

Dr Seungmin Jin
sedzhin@hse.ru

HSE

Learning Objectives

By the end of this session, you will be able to:

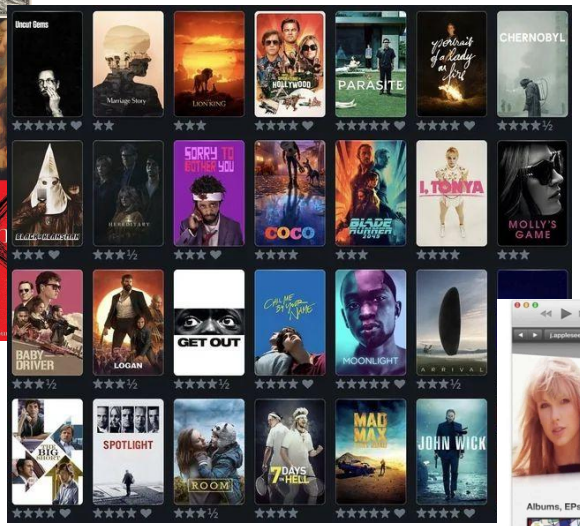
- Explain what **“content”** means in content-based filtering.
- Represent item content as feature vectors using metadata and text.
- Compute and interpret similarity scores, especially **cosine similarity**.
- Build an item-to-item or profile-based content recommender.

Example of Content-based filtering



Book Recommendation

Movie Recommendation



Music Recommendation



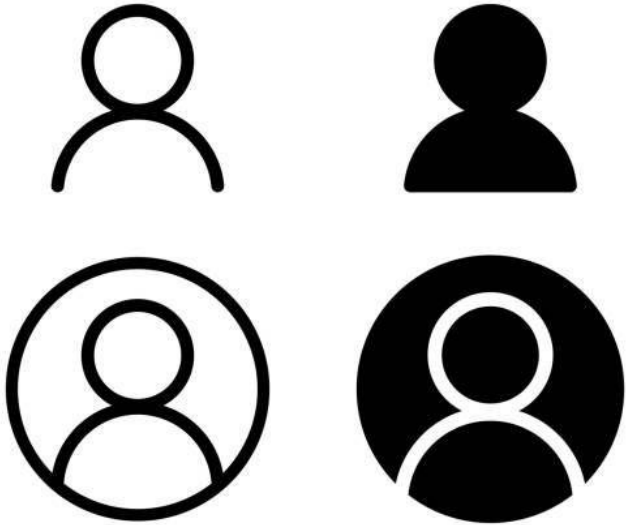
Entities

Content = Item (as features)



{ Features, Similarity, Metadata… }

User Profile = Item (as proxy)



{ Behavior, Ages, Preference …}

What Is “Content” in Content-Based Filtering?

Content originates from the item; profiles are reconstructed from interacted item content.

SOURCE: ITEM

Item Content

Everything describing the item itself

Genre Sci-Fi	**Tags** Space
Text Synopsis	**Image** Poster

Encoded Item Vector x_i

0.90	0.85	0.60	0.10
Sci-Fi	Space	Action	Drama

Raw metadata & multimodal features (Independent of other users)

interacts →

USER HISTORY

Interacted Items

Star Wars	`vector x_1`	w = 1.0
Dune	`vector x_2`	w = 1.0
Interstellar	`vector x_3`	w = 0.8

Observed content vectors with interaction weights

aggregate →

DERIVED: USER PROFILE

User Profile p_u

A proxy reconstructed from item content

$$p_u = \frac{\sum_{i \in I_u} w_{ui} x_i}{\sum_{i \in I_u} w_{ui}}$$

Same Feature Space as Items

0.81	0.87	0.45	0.16
Sci-Fi	Space	Action	Drama

Not raw user data (e.g., demographics).

It is an item-content-derived taste proxy.

KEY IDEA

Content-Based Filtering recommends candidates based entirely on item content, either directly matching active item features or via an aggregated user profile proxy.

How to create features of contents?

Aspect	Data-Driven Approach	Domain Expertise (Human Agents)
Definition	Uses historical data and ML to automatically identify important features.	Relies on expert knowledge and business intuition to manually select features.
Pros	Objective & discovers hidden patterns that experts might miss.	Fast & Simple, especially for new products with little data. Aligns with business logic.
Cons	Requires large, high-quality data. Can be complex and computationally expensive.	Subjective and may reflect personal biases or overlook non-obvious patterns.
Best For...	Optimizing mature systems with abundant user interaction data.	Initial system setup, new domains, or when data is scarce.

From Item Features to Vectors: Multimodal Feature Encoding

Transforming Raw Multimodal Metadata into a Unified Feature Vector: $\mathbf{x}_i \in \mathbb{R}^d$

RAW ITEM METADATA



Movie: Dune (2021)

- **Director:** Denis Villeneuve
- **Plot Summary:** "Paul Atreides travels to..."
- **Key Genres:** Sci-Fi, Adventure, Drama
- **User Tags:** #desert, #spice, #messiah

FEATURE ENCODING PIPELINE (MODALITY-SPECIFIC)

1. Tabular / Categorical Metadata $\mathbf{e}_{\text{meta}} \in \mathbb{R}^{d_1}$ **(Sparse/Dense)**

- Raw: Release Year, Rating, Age, Director
- **Model: One-Hot / Multi-Hot / Min-Max Scaling**

2. Unstructured Text (Plot / Synopsis) $\mathbf{e}_{\text{text}} \in \mathbb{R}^{d_2}$ **(Dense Semantic)**

- Raw: "Paul Atreides leads nomadic tribes..."
- **Model: Pretrained LLM / Transformer (Sentence-BERT)**

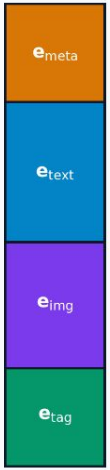
3. Visual Content (Poster / Keyframe) $\mathbf{e}_{\text{img}} \in \mathbb{R}^{d_3}$ **(Dense Visual)**

- Raw: High-res movie poster artwork & colors
- **Model: Vision Transformer / ResNet (CLIP Image)**

4. Social Tags & Keywords $\mathbf{e}_{\text{tag}} \in \mathbb{R}^{d_4}$ **(Weighted Vector)**

- Raw: #sandworm, #spice, #space_opera
- **Model: TF-IDF / Tag2Vec Entity Embeddings**

UNIFIED ITEM VECTOR $\mathbf{x}_i$



$$\mathbf{x}_i = [\mathbf{e}_{\text{meta}} \,\|\, \mathbf{e}_{\text{text}} \,\|\, \mathbf{e}_{\text{img}} \,\|\, \mathbf{e}_{\text{tag}}]$$

Concatenation or MLP Projection

Total Dimension: $d = \sum d_k$

Multimodal Fusion Schemes

- **Early Fusion (Concat):** Join raw features before input; simple but feature scale mismatch.
- **Late Fusion (Two-Tower):** Encode modalities separately, combine via Attention / Dot product. **Industry Standard in Modern RecSys.**

Ready for Downstream Recommendation:

$$\text{Score}(u, i) = \cos(\mathbf{p}_u, \mathbf{x}_i) = \frac{\mathbf{p}_u \cdot \mathbf{x}_i}{\|\mathbf{p}_u\| \|\mathbf{x}_i\|}$$

MANAGERIAL TAKEAWAY

1. Moving beyond One-Hot: Real-world items are rich in unstructured text (reviews, plot) and imagery (posters).

2. Dense Embeddings enable semantic matching (e.g. 'desert messiah' ≈ 'cosmic prophecy') that keyword matching misses.

Content-Based Filtering: The Core Philosophy

The 'Personal Shopper' Metaphor · Mapping Item Features to User Preferences

WHAT IS AN ITEM? (STRUCTURED ATTRIBUTES)

Example: Black Tailored Blazer (ID: #4092)

1. Categorical / Domain Attributes

Category: Formal	**Material: Wool (100%)**
Color: Onyx Black	**Cut: Slim Fit**

2. Textual Metadata (NLP / TF-IDF Features)

Title & Synopsis: 'Minimalist double-breasted tuxedo with satin lapel.'

Key Tokens: [minimalist, double-breasted, luxury, formal, bespoke]

3. Dense Feature Representation ($x_i \in \mathbb{R}^d$)

0.95	0.05	0.98	0.85	0.12
Formal	Casual	Dark	Luxury	Summer

Content = Everything describing the item itself (No other users needed)

User Likes or Buys Item x_i



HOW THE 'PERSONAL SHOPPER' THINKS

Step 1. User Taste Profiling (Inference)

"Since you loved this formal wool blazer with high luxury cut, you have high preference weights for: [Formal: 0.9, Dark: 0.95]."

Step 2. Similarity-Driven Filtering (Matching)

"Search catalog for items sharing these exact feature vectors, even if other customers never bought them together!"

Recommended Output:

- **Silk Lapel Dinner Vest (cos = 0.93)**
- **Italian Wool Slacks (cos = 0.89)**

(Zero reliance on other users' co-purchases!)

CORE PHILOSOPHY **You recommend based on WHAT the item is, not WHO ELSE bought it.**

Similarity Measures in Recommender Systems

Mathematical Foundations for Comparing Item Attributes & User Profiles

COSINE SIMILARITY

$$\cos(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x}^\top \mathbf{y}}{\|\mathbf{x}\|_2 \|\mathbf{y}\|_2}$$

Input Representation:

Dense vectors / TF-IDF weights

What it Captures:

Direction (Angle θ) only

RecSys Characteristics:

- Gold standard for CBF
- Length-normalized
- Invariant to document length
- Score bounded in [-1, 1]

Best for: Text & Weighted Feature Profiles

JACCARD SIMILARITY

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Input Representation:

Binary sets / Categorical tags

What it Captures:

Set overlap ratio (0 to 1)

RecSys Characteristics:

- Compares genre & tag sets
- Ignores feature frequencies
- Penalizes huge union sets
- Score bounded in [0, 1]

Best for: Multi-hot Tags & Unweighted Categoricals

DOT PRODUCT

$$\mathbf{x}^\top \mathbf{y} = \sum_{k=1}^{d} x_k y_k$$

Input Representation:

Dense learned embeddings (2-Tower)

What it Captures:

Direction + Magnitude

RecSys Characteristics:

- Fast MIPS computation
- Length-sensitive (Norm bias)
- Popular items dominate
- Unbounded score range

Best for: Neural Retrieval & Two-Tower MIPS

EUCLIDEAN DISTANCE

$$\|\mathbf{x} - \mathbf{y}\|_2 = \sqrt{\sum (x_k - y_k)^2}$$

Input Representation:

Continuous numerical attributes

What it Captures:

Absolute spatial distance

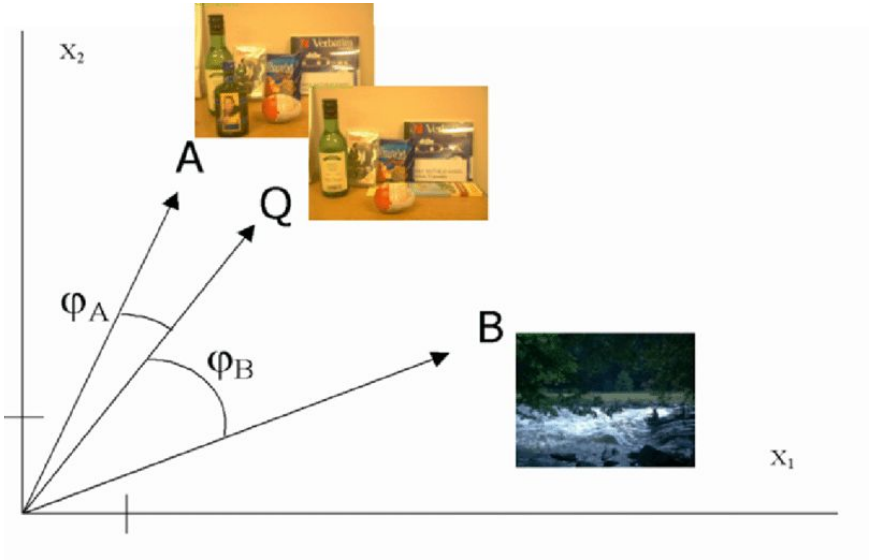
RecSys Characteristics:

- Distance metric (0 = identical)
- Sensitive to curse of dimensionality
- Requires conversion: 1 / (1 + d)
- Scale dependent

Best for: Low-Dimensional Continuous Features

PRACTICAL RULE — **Use Jaccard for binary tag sets · Use Cosine for TF-IDF & text · Use Dot Product for 2-Tower retrieval**

Cosine & Dot



$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}}$$

"Dot Product"

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix} = \begin{bmatrix} 58 & \end{bmatrix}$$

Why Cosine Similarity for Content-Based Filtering?

Eliminating Length Bias via Geometric Angle Alignment

THE PROBLEM: DOCUMENT LENGTH BIAS IN DOT PRODUCT

Item A (Short Synopsis / High Relevance)

Text: "Epic Sci-Fi space exploration and interstellar survival."

Counts: Sci-Fi: 1, Space: 1 $\longrightarrow$ $\|\mathbf{x}_A\|_2 = 1.41$ **(Small Norm)**

Item B (Long Wiki Article / Low Relevance)

Text: 500-word cast list, production trivia, studio notes...

Accumulates noisy word matches $\longrightarrow$ $\|\mathbf{x}_B\|_2 = 12.8$ **(Huge Norm)**

Why Unnormalized Dot Product Fails:

- $\mathbf{x}^\top \mathbf{y}$ grows monotonically with the total number of words.
- Verbose items score higher than concise, perfectly matched items.
- Dot product rewards verbosity, NOT true semantic preference!

Result: Fails to recommend what the user actually wants!

THE SOLUTION: COSINE NORMALIZATION IN ACTION

Feature Space: [Sci-Fi, Space, Action, Comedy]

$\mathbf{x}_{\text{Interstellar}} = [1, 1, 0, 0]$ $\quad \|\mathbf{x}\|_2 = \sqrt{1^2 + 1^2} = \sqrt{2} \approx 1.414$

$\mathbf{x}_{\text{The Martian}} = [1, 1, 1, 0]$ $\quad \|\mathbf{y}\|_2 = \sqrt{1^2 + 1^2 + 1^2} = \sqrt{3} \approx 1.732$

Dot Product: $\mathbf{x}^\top \mathbf{y} = (1 \times 1) + (1 \times 1) + 0 + 0 = 2$

$$\cos(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x}^\top \mathbf{y}}{\|\mathbf{x}\|_2 \|\mathbf{y}\|_2} = \frac{2}{\sqrt{2} \times \sqrt{3}} = \frac{2}{\sqrt{6}} \approx \mathbf{0.816}$$

Key Benefits for Content-Based Filtering:

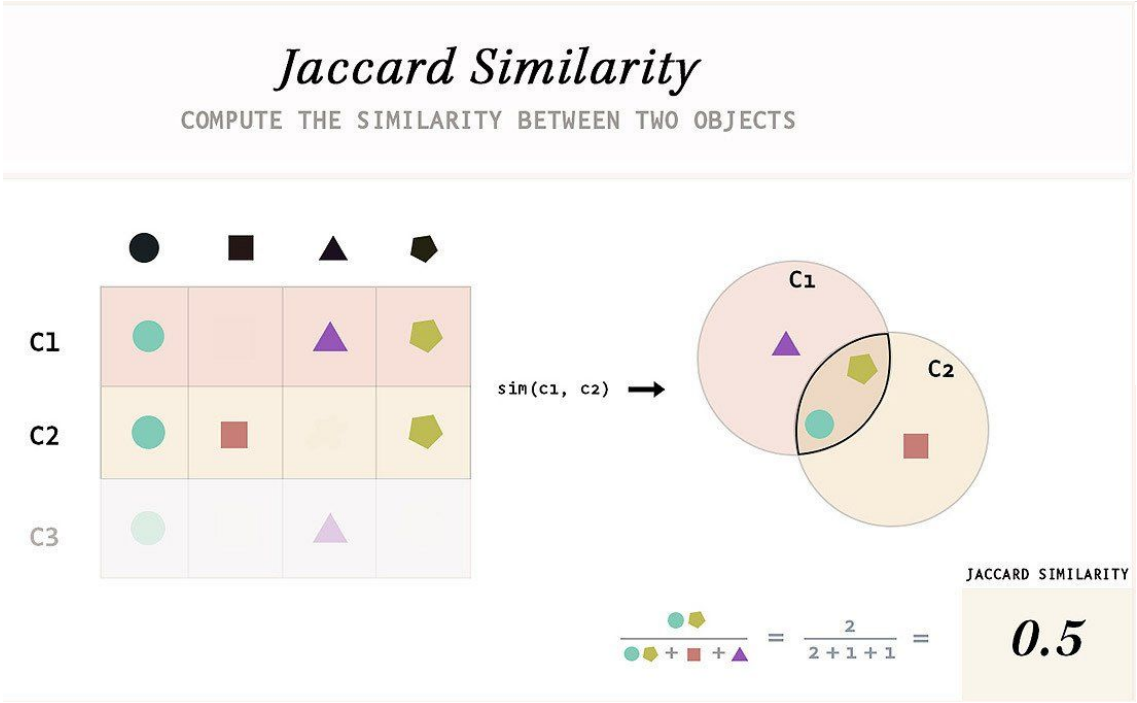
- Strictly bounded score: Always in [0, 1] for non-negative features.
- Scale-invariant: A 50-word synopsis matches a 500-word review fairly.
- Compares feature proportions (taste orientation), NOT word volume.

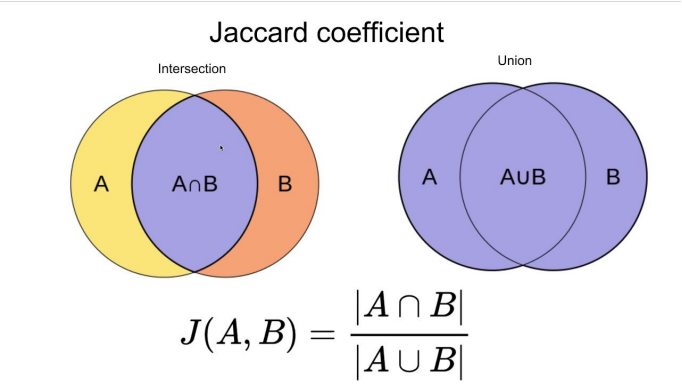
Normalized Score: 0.816

CORE TAKEAWAY

Cosine similarity measures shared feature ratios rather than total text length, effectively neutralizing length bias and serving as the industry standard for Content-Based Filtering.

Jaccard


Jaccard Similarity
COMPUTE THE SIMILARITY BETWEEN TWO OBJECTS
C1
C2
C3
sim(c1, c2)
C1
C2
JACCARD SIMILARITY
2 / 2+1+1 =
0.5


Jaccard coefficient
Intersection
A
A∩B
B
Union
A
A∪B
B
$J(A,B) = \frac{|A \cap B|}{|A \cup B|}$

Item-to-Item Recommendation Pipeline

Context-Driven · Finds candidates strictly aligned with the currently active item

1. ACTIVE ITEM (i_t)

Star Wars: Episode IV

Currently Active in User Session

Content Feature Vector x_t

0.90 Sci-Fi	**0.85** Space
0.60 Action	**0.10** Drama

Raw Metadata Representation (Genre, Synopsis, Tags)

2. COSINE SIMILARITY

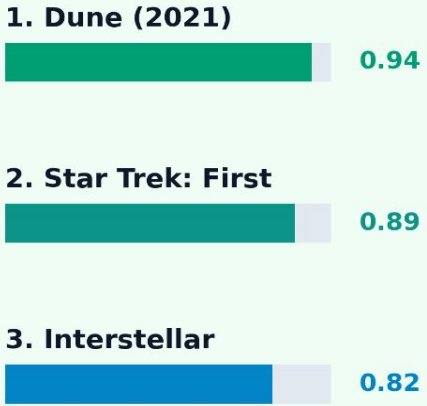
$$\cos(\mathbf{x}_t, \mathbf{x}_j) = \frac{\mathbf{x}_t^\top \mathbf{x}_j}{\|\mathbf{x}_t\|_2 \|\mathbf{x}_j\|_2}$$

Directional Alignment

- **Measures angle between vectors**
- **Length-normalized (no text bias)**
- **Fast nearest-neighbor retrieval**

Pairwise Catalog Matching score(j | i_t) in [-1, 1]

3. TOP-K RANKING



Immediate Next Recommendations (Related Candidates)

KEY PROPERTY **Zero user history required · Solves item cold-start instantly · Highly responsive to active session**

Profile-Based Recommendation Pipeline

History-Driven · Aggregates past interactions into a persistent user taste centroid

1. USER HISTORY (I_u)

- **Star Wars** — w_1=1.0
  `x_1=[0.9, 0.8, 0.5, 0.1]`
- **Dune** — w_2=1.0
  `x_2=[0.8, 0.9, 0.6, 0.0]`
- **Interstellar** — w_3=0.8
  `x_3=[0.7, 0.9, 0.2, 0.4]`

Aggregated Interaction History (Ratings, Clicks, Likes)

2. PROFILE ESTIMATION

$$\mathbf{p}_u = \frac{\sum_{i \in I_u} w_{ui} \mathbf{x}_i}{\sum_{i \in I_u} w_{ui}}$$

Profile Centroid Vector p_u

0.81 Sci-Fi	0.87 Space
0.45 Action	0.16 Drama

Weighted Center of Mass in Taste Space

3. TOP-K RANKING

$$\text{score}(u, j) = \cos(\mathbf{p}_u, \mathbf{x}_j)$$

1. **Blade Runner** — 0.92
2. **The Martian** — 0.88
3. **Arrival** — 0.81

Personalized Feed Candidates (Persistent Preferences)

KEY PROPERTY **Averages diverse actions into stable taste · High noise robustness against accidental clicks**

Profiling



Sci-fi, Cosmos, Battle, Superpower

The user liked Star Wars

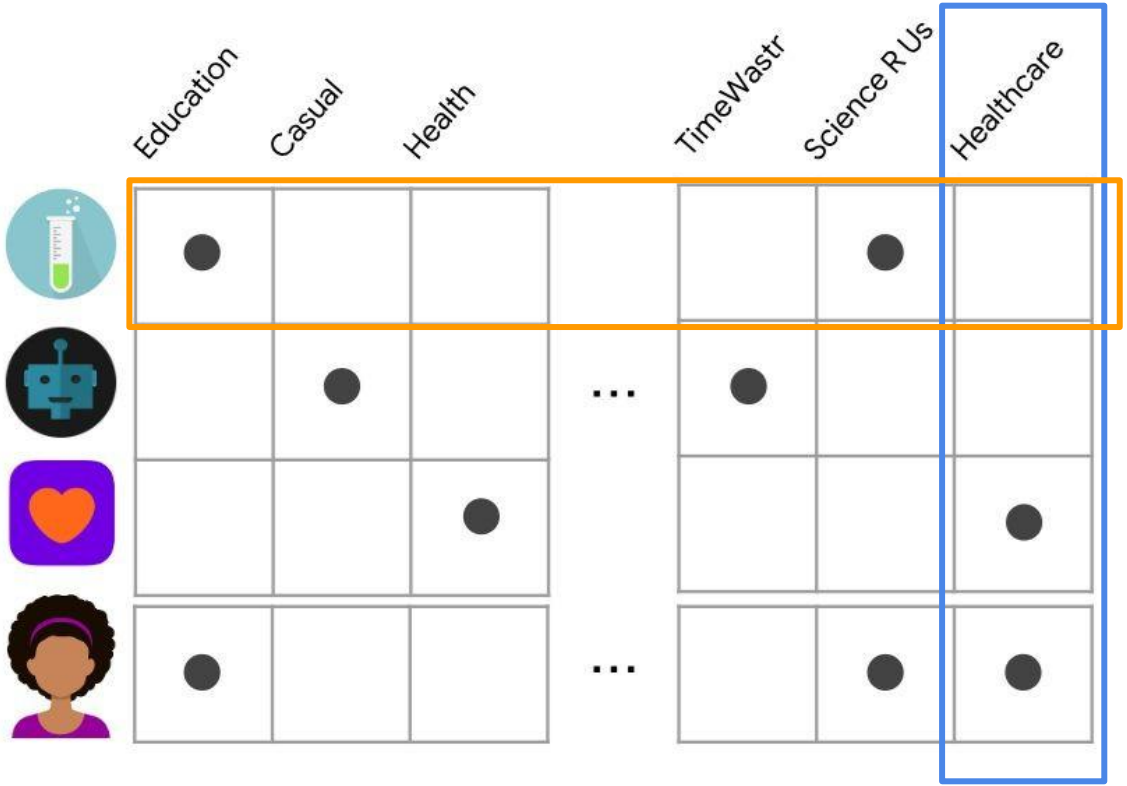
The user would like Dune too!

Features

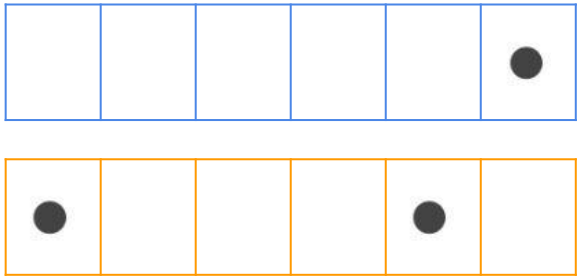


Sci-fi, Cosmos, Battle, Superpower

Content Matrix



Profile Matrix



Average

0.5				0.5	0.5

Item-to-Item vs. Profile-Based: The Context Dilemma

Immediate Session Window (Toy Story) vs. Long-Term History Centroid (Sci-Fi Collection)

USER WATCH HISTORY (Time Flow: Past ——> Present)



Star Wars
Sci-Fi (1977)
t-3 (Watched)

→



Dune
Sci-Fi (2021)
t-2 (Liked)

→



Interstellar
Sci-Fi (2014)
t-1 (5-Star)

→



Toy Story 5
Animation (2026)
t (NOW / Active)

⚡ Sudden Context Shift!
User plays a cartoon for their child right now

MODEL A: ITEM-TO-ITEM (CONTEXT WINDOW = 1)

Input Mechanism: Sees ONLY Active Item (Toy Story 5)



Finding Nemo
Animation · 2003
cos = 0.95
High Session Match



Shrek
Animation · 2001
cos = 0.91
High Session Match

PRO **Immediate intent captured:**
Recommends cartoons for the child right away!

CON **Rabbit Hole / Narrow Bias:**
Core Sci-Fi taste is completely ignored and lost.

MODEL B: PROFILE-BASED (AVERAGED CENTROID)

Input Mechanism: 75% Sci-Fi + 25% Animation => Sci-Fi Centroid



Blade Runner
Sci-Fi · 2017
cos = 0.92
High Profile Match



Arrival
Sci-Fi · 2016
cos = 0.88
High Profile Match

PRO **Noise-proof Global Taste:**
1 cartoon click won't ruin core Sci-Fi identity.

CON **Session Lag (Context Blind):**
Recommends dark Sci-Fi while sitting with a child!

INDUSTRIAL SOLUTION

Two-Stage Hybrid: Retrieve broad candidates with Profile Centroid (Global Taste), then Re-rank top candidates using Active Session Window & Sequential Transformers.

Movie Recommender System

[MovieLens 100K Dataset](#)

This data set consists of: * 100,000 ratings (1-5) from 943 users on 1682 movies. * Each user has rated at least 20 movies.

movies.csv (494.43 kB)

Detail | Compact | Column

movieId	title	genres
1 … 194k	9737 unique values	Drama 11% Comedy 10% Other (7743) 79%
1	Toy Story (1995)	Adventure|Animation|Children|Comedy|Fantasy
2	Jumanji (1995)	Adventure|Children|Fantasy
3	Grumpier Old Men (1995)	Comedy|Romance
4	Waiting to Exhale (1995)	Comedy|Drama|Romance
5	Father of the Bride Part II (1995)	Comedy
6	Heat (1995)	Action|Crime|Thriller

ratings.csv (2.48 MB)

Detail | Compact | Column

About this file

This file does not have a description yet.

userId	movieId	rating
1 … 610	1 … 194k	0.5 …
1	1	4.0
1	3	4.0
1	6	4.0
1	47	5.0
1	50	5.0
1	70	3.0
1	101	5.0
1	110	4.0
1	151	5.0

LLM Workflow

- Brainstorming
- Readme-Driven Development
- Plan based on the Readme
- Thinking model coding prompts
- LLM writes code and debugs tests

Movie Recommender System

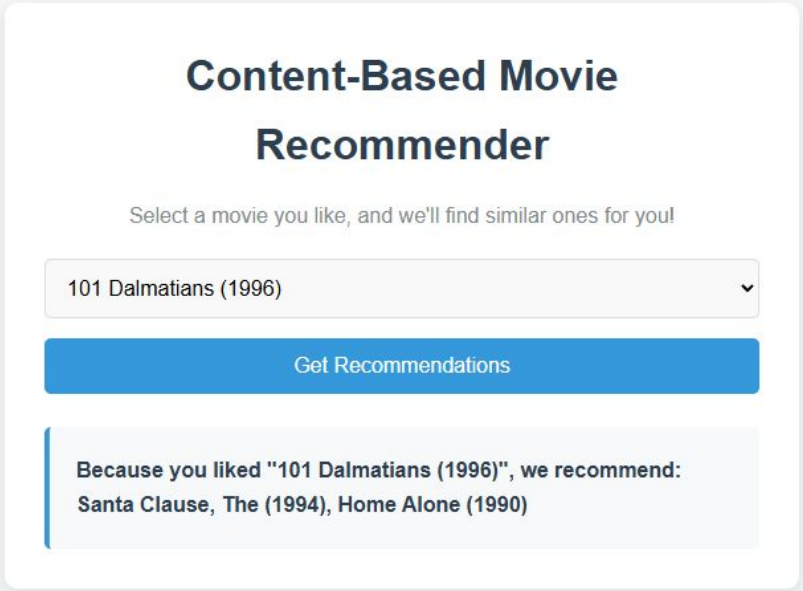
1. Implementation Task

- **Repository**: dryjins/RecSys-LLMs/tree/main/week2
- **Cosine Similarity**: Replace naive matching with cosine similarity
- **Profile-Based Logic**: Build a user profile vector by averaging multiple movies (e.g., 3 watched titles) and recommend Top-5 items

2. Business & Algorithmic Analysis

- **Item-to-Item vs. Profile-Based**: Compare Top-5 recommendations from a single active item vs. an aggregated profile
- **Bias Mitigation**: Analyze why cosine normalization prevents multi-genre blockbusters from dominating recommendations
- **Catalog Discovery**: Evaluate which approach uncovers long-tail items better to prevent user fatigue and support retention

Content-Based Movie Recommender



MovieLens 100K Dataset